

HHCV COURSE of Artificial Neural Networks

Contents

- 1 Deepseek-V1
- 2 Deepseek-V2

Deepseek-V1

原始文献：
DeepSeek LLM Scaling Open-Source Language Models with Longtermism, 86 authors, arXiv:2401.02954v1

Table 2 | Detailed specs of **DeepSeek LLM(V1)** family of models.(2401.02954)

Params (billion) 十亿	n_{layers}	d_{model}	n_{heads}	$n_{\text{kv_heads}}$	Context Length	Sequence Batch Size	Learning Rate	Tokens (trillion) 万亿
7B	30	4096	32	32	4096	2304	4.2e-4	2.0T
67B	95	8192	64	8	4096	4608	3.2e-4	2.0T

解释：
input length
input length **输入序列长度**，输入给大模型的样本序列（**一个训练样本**是一对 token 序列）的 token 数量。

context length
context length **上下文长度**，大模型能处理的**最大输入序列长度**。

sequence batch size
sequence batch size **序列样本批量**，一个批次训练，输入给大模型的样本序列的数量。解释模型算法时，通常是针对一个训练样本，而实际训练时，是按批次进行训练的。

Deepseek-V2

原始文献:

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, DeepSeek-AI, arXiv:2405.04434v5

2. Architecture(2405.04434)

By and large, **DeepSeek-V2** is still in the Transformer architecture (Vaswani et al., 2017), where each Transformer block consists of an attention module and a Feed-Forward Network (FFN).

However, for both the attention module and the FFN, we design and employ innovative architectures.

For attention, we **design MLA**, which utilizes low-rank key-value joint compression to eliminate the bottleneck of inference-time key-value cache, thus supporting efficient inference.

For FFNs, we **adopt the DeepSeekMoE** architecture (Dai et al., 2024), a high-performance MoE architecture that enables training strong models at an economical cost.

An illustration of the architecture of DeepSeek-V2 is presented in Figure 2, and we will introduce the details of MLA and DeepSeekMoE in this section.

For other tiny details (e.g., layer normalization and the activation function in FFNs), unless specifically stated, DeepSeek-V2 follows the settings of DeepSeek 67B (DeepSeek-AI, 2024, 2401.02954).

2.1. Multi-Head Latent Attention: Boosting Inference Efficiency

Conventional Transformer models usually adopts **Multi-Head Attention (MHA)** (Vaswani et al., 2017), but during generation, its heavy Key-Value (KV) cache will become the bottleneck that limit the inference efficiency.

In order to reduce the KV cache, **Multi-Query Attention (MQA)** (Shazeer, 2019) and Grouped-Query Attention (GQA) (Ainslie et al., 2023) are proposed.

They require a smaller magnitude of KV cache, but their performance does not match MHA (we provide the ablation of MHA, GQA and MQA in Appendix D.1).

For DeepSeek-V2, we design an innovative attention mechanism called **Multi-head Latent Attention (MLA)**.

Equipped with low-rank key-value joint compression, MLA achieves better performance than MHA,

but requires a significantly smaller amount of KV cache.

We introduce its architecture in the following, and also provide a comparison between MLA and MHA in Appendix D.2.

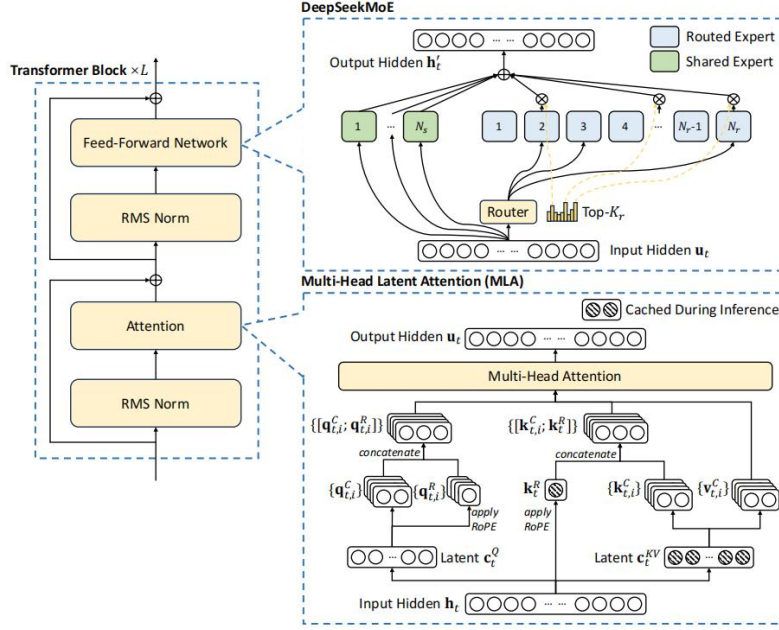


Figure 2 | Illustration of the architecture of DeepSeek-V2. MLA ensures efficient inference by significantly reducing the KV cache for generation, and DeepSeekMoE enables training strong models at an economical cost through the sparse architecture.

2.1.1. Preliminaries: Standard Multi-Head Attention

We first introduce the standard MHA mechanism as background. Let d be the embedding dimension, n_h be the number of attention heads, d_h be the dimension per head, and $\mathbf{h}_t \in \mathbb{R}^d$ be the attention input of the t -th token at an attention layer.

Standard MHA first produces $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t \in \mathbb{R}^{d_h n_h}$ through three matrices $W^Q, W^K, W^V \in \mathbb{R}^{d_h n_h \times d}$, respectively:

$$\begin{aligned}\mathbf{q}_t &= W^Q \mathbf{h}_t \\ \mathbf{k}_t &= W^K \mathbf{h}_t \\ \mathbf{v}_t &= W^V \mathbf{h}_t\end{aligned}$$

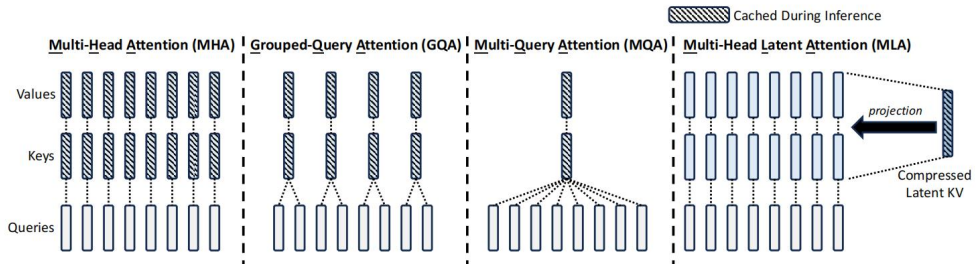


Figure 3 | Simplified illustration of Multi-Head Attention (MHA), Grouped-Query Attention (GQA), Multi-Query Attention (MQA), and Multi-head Latent Attention (MLA). Through jointly compressing

the keys and values into a latent vector, MLA significantly reduces the KV cache during inference.

Then, $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$ will be sliced into n_h heads for the multi-head attention computation:

$$\begin{aligned} [\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] &= \mathbf{q}_t \\ [\mathbf{k}_{t,1}; \mathbf{k}_{t,2}; \dots; \mathbf{k}_{t,n_h}] &= \mathbf{k}_t \\ [\mathbf{v}_{t,1}; \mathbf{v}_{t,2}; \dots; \mathbf{v}_{t,n_h}] &= \mathbf{v}_t \\ \mathbf{o}_{t,i} &= \sum_{j=1}^t \text{softmax}_j \left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_{j,i}}{\sqrt{d_h}} \right) \mathbf{v}_{j,i} \\ \mathbf{u}_t &= W^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}] \end{aligned}$$

Where $\mathbf{q}_{t,i}, \mathbf{k}_{t,i}, \mathbf{v}_{t,i} \in \mathbb{R}^{d_h}$ denote the query, key, and value of the i -th attention head, respectively; $W^O \in \mathbb{R}^{d \times d_h n_h}$ denotes the output projection matrix.

During inference, all keys and values need to be cached to accelerate inference, so MHA needs to cache $2n_h d_h l$ elements for each token.

In model deployment, this heavy KV cache is a large bottleneck that limits the maximum batch size and sequence length.

2.1.2. Low-Rank Key-Value Joint Compression

The core of MLA is the low-rank joint compression for keys and values to reduce KV cache:

$$\begin{aligned} \mathbf{c}_t^{KV} &= W^{DKV} \mathbf{h}_t \\ \mathbf{k}_t^C &= W^{UK} \mathbf{c}_t^{KV} \\ \mathbf{v}_t^C &= W^{UV} \mathbf{c}_t^{KV} \end{aligned}$$

where $\mathbf{c}_t^{KV} \in \mathbb{R}^{d_c}$ is the **compressed latent vector for keys and values**;

$d_c (\ll d_h n_h)$ denotes the **KV compression dimension**;

$W^{DKV} \in \mathbb{R}^{d_c \times d}$ is the **down-projection matrix**;

and $W^{UK}, W^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$ are the **up-projection matrices** for keys and values, respectively.

During inference, MLA only needs to cache $\mathbf{c}_t^{KV}$, so its KV cache has only $d_c l$ elements, where l denotes the number of layers.

In addition, during inference, since W^{UK} can be absorbed into W^Q , and W^{UV} can be absorbed into W^O , we even do not need to compute keys and values out for attention.

Figure 3 intuitively illustrates how the KV joint compression in MLA reduces the KV cache.

Moreover, in order to reduce the activation memory during training, we also perform low-rank compression for the queries, even if it cannot reduce the KV cache:

$$\mathbf{c}_t^Q = W^{DQ} \mathbf{h}_t$$

$$\mathbf{q}_t^C = W^{UQ} \mathbf{c}_t^Q$$

where $\mathbf{c}_t^Q \in \mathbb{R}^{d_c'}$ is the **compressed latent vector for queries**;

$d_c' (\ll d_h n_h)$ denotes the **query compression dimension**;

and $W^{DQ} \in \mathbb{R}^{d_c' \times d}$, $W^{UQ} \in \mathbb{R}^{d_h n_h \times d_c'}$ are the **down-projection** and **up-projection** matrices for queries, respectively.

2.1.3. Decoupled Rotary Position Embedding

Following DeepSeek 67B (DeepSeek-AI, 2024), we intend to use the **Rotary Position Embedding (RoPE)** (Su et al., 2024) for DeepSeek-V2.

However, RoPE is incompatible with low-rank KV compression.

To be specific, RoPE is position-sensitive for both keys and queries.

If we apply RoPE for the keys $\mathbf{k}_t^C$, W^{UK} in Equation 10 ($\mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV}$) will be coupled with a position-sensitive RoPE matrix.

In this way, W^{UK} cannot be absorbed into W^Q any more during inference, since a RoPE matrix related to the currently generating token will lie between W^Q and W^{UK} and matrix multiplication does not obey a commutative law.

As a result, we must recompute the keys for all the prefix tokens during inference, which will significantly hinder the inference efficiency.

As a solution, we propose the **decoupled RoPE** strategy that uses **additional multi-head queries**

$\mathbf{q}_{t,i}^R \in \mathbb{R}^{d_h^R}$ and a shared key $\mathbf{k}_t^R \in \mathbb{R}^{d_h^R}$ to carry RoPE, where d_h^R denotes the **per-head dimension** of the decoupled queries and key.

Equipped with the decoupled RoPE strategy, MLA performs the following computation:

$$[\mathbf{q}_{t,1}^R; \mathbf{q}_{t,2}^R; \dots; \mathbf{q}_{t,n_h}^R] = \mathbf{q}_t^R = \text{RoPE}(W^{QR} \mathbf{c}_t^Q)$$

$$\mathbf{k}_t^R = \text{RoPE}(W^{KR} \mathbf{h}_t)$$

$$\mathbf{q}_{t,i} = [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R]$$

$$\mathbf{k}_{t,i} = [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R]$$

$$\mathbf{o}_{t,i} = \sum_{j=1}^t \text{softmax}_j \left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}} \right) \mathbf{v}_{j,i}^C$$

$$\mathbf{u}_t = W^O[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}]$$

where $W^{QR} \in \mathbb{R}^{d_h^R n_h \times d_c'}$ and $W^{KR} \in \mathbb{R}^{d_h^R \times d}$ are matrices to produce the decouples queries and key, respectively;

$\text{RoPE}(\cdot)$ denotes the operation that applies RoPE matrices; and $[\cdot; \cdot]$ denotes the concatenation operation.

During inference, the decoupled key should also be cached.

Therefore, DeepSeek-V2 requires a total KV cache containing $(d_c + d_h^R)l$ elements.

In order to demonstrate the complete computation process of MLA, we also organize and provide its full formulas in Appendix C.

2.1.4. Comparison of Key-Value Cache

We demonstrate a comparison of the KV cache per token among different attention mechanisms in Table 1. MLA requires only a small amount of KV cache, equal to GQA with only 2.25 groups, but can achieve stronger performance than MHA.

2.2. DeepSeekMoE: Training Strong Models at Economical Costs

2.2.1. Basic Architecture

For FFNs, we employ the **DeepSeekMoE** architecture (Dai et al., 2024). (**MoE, Mixture-of-Experts**)

DeepSeekMoE has two key ideas: segmenting experts into finer granularity for higher expert specialization and more accurate knowledge acquisition, and isolating some shared experts for mitigating knowledge redundancy among routed experts.

With the same number of activated and total expert parameters, DeepSeekMoE can outperform conventional MoE architectures like GShard (Lepikhin et al., 2021) by a large margin.

Let $\mathbf{u}_t$ be the FFN input of the t -th token, we compute the FFN output $\mathbf{h}'_t$ as follows:

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t)$$

$$g_{i,t} = \begin{cases} s_{i,t} & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r) \\ 0 & \text{otherwise} \end{cases}$$

$$s_{i,t} = \text{softmax}_i(\mathbf{u}_t^T \mathbf{e}_i)$$

where N_s and N_r denote the numbers of **shared experts** and **routed experts**, respectively;

$\text{FFN}_i^{(s)}(\cdot)$ and $\text{FFN}_i^{(r)}(\cdot)$ denote the i -th shared expert and the i -th routed expert, respectively;

K_r denotes the number of activated routed experts;

$g_{i,t}$ is the gate value for the i -th expert;

$s_{i,t}$ is the token-to-expert affinity;

$\mathbf{e}_i$ is the centroid of the i -th routed expert in this layer;

and $\text{Topk}(\cdot, K)$ denotes the set comprising K highest scores among the affinity scores calculated for the t -th token and all routed experts.

2.2.2. Device-Limited Routing

We design a device-limited routing mechanism to bound MoE-related communication costs.

When expert parallelism is employed, the routed experts will be distributed across multiple devices.

For each token, its MoE-related communication frequency is proportional to the number of devices covered by its target experts.

Due to the fine-grained expert segmentation in DeepSeekMoE, the number of activated experts can be large, so the MoE-related communication will be more costly if we apply expert parallelism.

For DeepSeek-V2, beyond the naive top-K selection of routed experts, we additionally ensure that the target experts of each token will be distributed on at most M devices.

To be specific, for each token, we first select M devices that have experts with the highest affinity

scores in them.

Then, we perform top-K selection among experts on these M devices.

In practice, we find that when $M \geq 3$, the device-limited routing can achieve a good performance roughly aligned with the unrestricted top-K routing.

2.2.3. Auxiliary Loss for Load Balance

We take the load balance into consideration for automatically learned routing strategies.

Firstly, unbalanced load will raise the risk of routing collapse (Shazeer et al., 2017), preventing some experts being fully trained and utilized.

Secondly, when expert parallelism is employed, unbalanced load will diminish computation efficiency.

During the training of DeepSeek-V2, we design three kinds of auxiliary losses, for controlling expert-level load balance ($\mathcal{L}_{\text{ExpBal}}$), device-level load balance ($\mathcal{L}_{\text{DevBal}}$), and communication balance ($\mathcal{L}_{\text{CommBal}}$), respectively.

Expert-Level Balance Loss.

We use an expert-level balance loss (Fedus et al., 2021; Lepikhin et al., 2021) to mitigate the risk of routing collapse:

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N_r} f_i P_i$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^T \text{Indicator}(\text{Token } t \text{ selects Expert } i)$$

$$P_i = \frac{1}{T} \sum_{t=1}^T s_{i,t}$$

where α_1 is a hyper-parameter called expert-level balance factor; $\text{Indicator}(\cdot)$ denotes the indicator function; and T denotes the number of tokens in a sequence.

Device-Level Balance Loss.

In addition to the expert-level balance loss, we additionally design a device-level balance loss to ensure balanced computation across different devices.

In the training process of DeepSeek-V2, we partition all routed experts into D groups $\{\mathcal{E}_1, \mathcal{E}_2, \dots, \mathcal{E}_D\}$, and deploy each group on a single device.

The device-level balance loss is computed as follows:

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^D f'_i P'_i$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j$$

where α_2 is a hyper-parameter called device-level balance factor.

Communication Balance Loss.

Finally, we introduce a communication balance loss to ensure that the communication of each device is balanced.

Although the device-limited routing mechanism guarantees that the sending communication of each device is bounded, if a certain device receives more tokens than other devices, the practical communication efficiency will also be affected.

In order to mitigate this issue, we design a communication balance loss as follows:

$$\mathcal{L}_{\text{CommBal}} = \alpha_3 \sum_{i=1}^D f''_i P''_i$$

$$f''_i = \frac{D}{MT} \sum_{t=1}^T \text{Indicator}(\text{Token } t \text{ is sent to Device } i)$$

$$P''_i = \sum_{j \in \mathcal{E}_i} P_j$$

where α_3 is a hyper-parameter called communication balance factor.

The device-limited routing mechanism operates on the principle of ensuring that each device transmits at most MT hidden states to other devices.

Simultaneously, the communication balance loss is employed to encourage each device to receive around MT hidden states from other devices.

The communication balance loss guarantees a balanced exchange of information among devices, promoting efficient communications.

2.2.4. Token-Dropping Strategy

While balance losses aim to encourage a balanced load, it is important to acknowledge that they cannot guarantee a strict load balance.

In order to further mitigate the computation wastage caused by unbalanced load, we introduce a device-level token-dropping strategy during training.

This approach first computes the average computational budget for each device, which means that the capacity factor for each device is equivalent to 1.0.

Then, inspired by Riquelme et al. (2021), we drop tokens with the lowest affinity scores on each device until reaching the computational budget.

In addition, we ensure that the tokens belonging to approximately 10% of the training sequences will never be dropped.

In this way, we can flexibly decide whether to drop tokens during inference according to the efficiency requirements, and always ensure consistency between training and inference.

3. Pre-Training

3.1. Experimental Setups

3.1.1. Data Construction

While maintaining the same data processing stages as for DeepSeek 67B (DeepSeek-AI, 2024), we extend the amount of data and elevate the data quality.

In order to enlarge our pre-training corpus, we explore the potential of the internet data and optimize our cleaning processes, thus recovering a large amount of mistakenly deleted data.

Moreover, we incorporate more Chinese data, aiming to better leverage the corpus available on the Chinese internet.

In addition to the amount of data, we also focus on the data quality.

We enrich our pre-training corpus with high-quality data from various sources, and meanwhile improve the quality-based filtering algorithm.

The improved algorithm ensures that a large amount of non-beneficial data will be removed, while the valuable data will be mostly retained.

In addition, we filter out the contentious content from our pre-training corpus to mitigate the data bias introduced from specific regional cultures.

A detailed discussion about the influence of this filtering strategy is presented in Appendix E.

We adopt the same tokenizer as used in DeepSeek 67B, which is built based on the **Byte-level Byte-Pair Encoding (BBPE)** algorithm and has a **vocabulary size of 100K**.

Our tokenized pretraining **corpus contains 8.1T tokens**, where Chinese tokens are approximately 12% more than English ones.

Table | Detailed specs of **DeepSeek (V2)** family of models.(2405.04434)

Params (billion) 十亿	n_{layers}	d_{model}	n_{heads}	$n_{\text{kv_heads}}$	Context Length	Sequence Batch Size	Learning Rate	Tokens (trillion) 万亿
236B	60	5120	128	64	128k	9216	2.4e-4	8.1T

3.1.2. Hyper-Parameters

Model Hyper-Parameters.

We set the number of **Transformer layers to 60** and the **hidden dimension to 5120**.

All learnable parameters are randomly initialized with a standard deviation of 0.006.

In MLA, we set the number of **attention heads n_h to 128** and the **per-head dimension d_h to 128**.

The **KV compression dimension d_c is set to 512**, and the **query compression dimension d'_c is set to 1536**.

For the **decoupled queries and key**, we set the **per-head dimension d_h^R to 64**.

Following Dai et al. (2024), we substitute all FFNs except for the first layer with MoE layers.

Each MoE layer consists of **2 shared experts** and **160 routed experts**, where the intermediate hidden dimension of each expert is 1536.

Among the routed experts, 6 experts will be activated for each token.

In addition, the low-rank compression and fine-grained expert segmentation will impact the output scale of a layer.

Therefore, in practice, we employ additional RMS Norm layers after the compressed latent vectors, and multiply additional scaling factors at the width bottlenecks (i.e., the compressed latent vectors and the intermediate hidden states of routed experts) to ensure stable training.

Under this configuration, DeepSeek-V2 comprises **236B total parameters**, of which 21B are activated for each token.

Training Hyper-Parameters.

We employ the AdamW optimizer (Loshchilov and Hutter, 2017) with hyper-parameters set to $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\text{weight_decay} = 0.1$.

The learning rate is scheduled using a warmup-and-step-decay strategy (DeepSeek-AI, 2024).

Initially, the learning rate linearly increases from 0 to the maximum value during the first 2K steps.

Subsequently, the learning rate is multiplied by 0.316 after training about 60% of tokens, and again by 0.316 after training about 90% of tokens.

The maximum **learning rate is set to 2.4×10^{-4}** , and the gradient clipping norm is set to 1.0.

We also use a **batch size** scheduling strategy, where the batch size is gradually increased from 2304 to 9216 in the training of the first 225B tokens, and then keeps **9216** in the remaining training.

We set the **maximum sequence length to 4K**, and train DeepSeek-V2 on **8.1T tokens**.

We leverage **pipeline parallelism** to deploy different layers of a model on different devices, and for each layer, the routed experts will be uniformly deployed on 8 devices ($D=8$).

As for the device-limited routing, each token will be sent to at most 3 devices ($M=3$).

As for balance losses, we set α_1 to 0.003, α_2 to 0.05, and α_3 to 0.02.

We employ the token-dropping strategy during training for acceleration, but do not drop any tokens for evaluation.

3.1.3. Infrastructures

DeepSeek-V2 is trained based on the HAI-LLM framework (High-flyer, 2023), an efficient and light-weight training framework developed internally by our engineers.

It employs a 16-way zero-bubble pipeline parallelism (Qi et al., 2023), an 8-way expert parallelism (Lepikhin et al., 2021), and ZeRO-1 data parallelism (Rajbhandari et al., 2020).

Given that DeepSeek-V2 has relatively few activated parameters, and a portion of the operators are recomputed to save activation memory, it can be trained without the necessity of tensor parallelism, thereby decreasing the communication overhead.

Moreover, in order to further improve the training efficiency, we overlap the computation of shared experts with the expert parallel all-to-all communication.

We also customize faster CUDA kernels for communications, routing algorithms, and fused linear computations across different experts.

In addition, MLA is also optimized based on an improved version of FlashAttention-2 (Dao, 2023).

We conduct all experiments on a cluster equipped with NVIDIA H800 GPUs.

Each node in the H800 cluster contains 8 GPUs connected using NVLink and NVSwitch within nodes.

Across nodes, InfiniBand interconnects are utilized to facilitate communications.

3.1.4. Long Context Extension

After the initial pre-training of DeepSeek-V2, we employ YaRN (Peng et al., 2023) to extend the default **context window length from 4K to 128K**.

YaRN was specifically applied to the **decoupled shared key k_t^R** as it is responsible for carrying RoPE (Su et al., 2024).

For YaRN, we set the scale s to 40, α to 1, β to 32, and the target maximum context length to 160K.

Under these settings, we can expect the model to respond well for a context length of 128K.

Slightly diverging from original YaRN, due to our distinct attention mechanism, we adjust the length scaling factor to modulate the attention entropy.

The factor $\sqrt{t}$ is computed as $\sqrt{t} = 0.0707 \ln s + 1$, aiming at minimizing the perplexity.

We additionally train the model for 1000 steps, with a **sequence length of 32K** and a **batch size of 576** sequences.

Although the training is conducted solely at the sequence length of 32K, the model still demonstrates robust performance when being evaluated at a **context length of 128K**.

As shown in Figure 4, the results on the “Needle In A Haystack” (NIAH) tests indicate that DeepSeek-V2 performs well across all context window lengths up to 128K.